

CORS endpoint echoes unsafe JSON / Potential XSS

CORS endpoint echoes unsafe JSON / Potential XSS - ebenoist, GitHub.

- Published: 2020-04-20
- Original: <<https://github.com/assembler/attachinary/issues/172>>
- Preserved from: <https://github.com/assembler/attachinary/issues/172> (github-api) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

CORS endpoint echoes unsafe JSON / Potential XSS

- Repository: assembler/attachinary
- Opened by: ebenoist
- Opened: 2020-04-20
- State: open

Body

The `/cors` endpoint, as configured currently echo's any params passed to the `/attachinary/cors` endpoint. This means that unscrupulous users can use this endpoint to trick JavaScript code into thinking that the content coming back is from a safe endpoint.

Example

Let's say I have a route in my SPA that looks like this:

`https://mysite.com/products/my-great-product`

An unscrupulous user can trick someone into clicking on a link that looks like this:

`https://mysite.com/products/../../../../attachinary/cors?title="<script>alert('foo')</script>"`

If my JavaScript code interprets everything after the `products/` as the slug for my application, I may naively make a call with that value:

```
fetch("https://mysite.com/api/products/" + slug)
```

Which will resolve to `https://mysite.com/attachinary/cors?description="<script>alert('foo')</script>"` and return whatever was in the params.

```
{
  "description": "<script>alert('foo')</script>"
}
```

Now if my application, receiving that data renders that into the document without first scrubbing, the result will be a successful XSS attack. Its not unreasonable to think that my application should be returning html safe strings, but the combination of the path traversal hack and the `/attachinary/cors` endpoint blindly echo-ing input creates an easy vector for XSS.

Is there a reason the cors endpoint should be echoing back params that should be considered unsafe?

Archived from <https://github.com/assembler/attachinary/issues/172>. Rendered offline from the local Markdown copy.